

Kubernetes

Need for Kubernetes

Containerization & Docker gained popularity pretty fast when it was introduced due to its features like a lightweight OS, portable environments and sharable isolated host kernel.

However, they came with their own limitations, such as:

1. Containers are only able to run on a single host
2. Containers are ephemeral and thus need manual intervention for a restart
3. Containers were not autoscalable
4. Containers didn't come with internal features like Load balancing, firewalls, and API gateways

Overall, there was no solid way to orchestrate these containers.

Introduction to Kubernetes

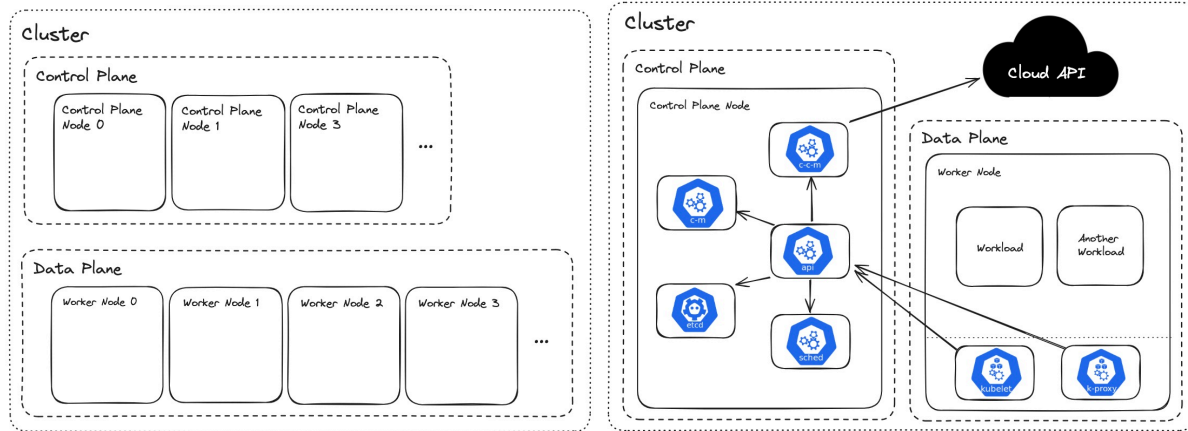
As a solution for the limitation of Containerization, Kubernetes was introduced. Kubernetes is a container orchestration platform that transforms the way applications are deployed, scaled, and managed in cloud environments.

Kubernetes supports resources that provide features like:

1. Autoscaling
2. Autohealing
3. Support of multiple hosts using nodes
4. Firewall, load-balancing, and API gateways

Architecture

A Kubernetes architecture consists of Clusters either under custom namespaces or default namespaces. A namespace helps to define a group under which a Kubernetes cluster runs.



A Kubernetes cluster consists of two planes/nodes

1. **Control Plane** (Master node): It manages the cluster and makes global decisions for the cluster
2. **Data Plane** (Worker nodes): It runs the actual application workload under multiple worker nodes

Control Plane components:

1. API server:
 - a. Central hub of Kubernetes APIs
 - b. Manages API requests that go in and out of the cluster
2. Scheduler: Decides which node a pod should run on
3. etcd: Key-value store that keeps cluster data and configs (source of truth)
4. Controller manager:
 - a. Manages the desired cluster state
 - b. Also manages controllers like ReplicaSets
5. Cloud Controller manager: Supports and helps to connect with the desired Cloud provider (AWS, Azure, etc.)

Data plane components:

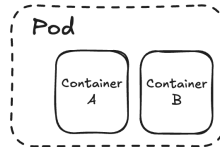
1. Kubelet:
 - a. Manages a pod lifecycle on each node
 - b. Reports back to the Master node
2. Kube-proxy
 - a. Handles networking rules for the nodes
 - b. Manages load balancing through iptables
3. Container-runtime
 - a. Executes the containers
 - b. Provides support to run containers on Docker shim, CRI-O, and containerd

Kubernetes Resources

To manage the applications and implement the features, Kubernetes provides support for multiple resources.

Pods

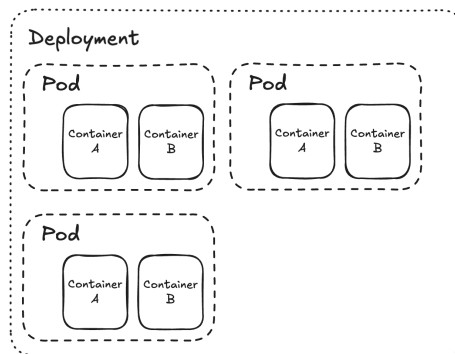
1. A pod is the smallest deployable unit that wraps one or more containers



2. Pods are managed over a cluster of nodes
3. It enables fault isolation and resource distribution
4. Pods are ephemeral
5. Written in a YAML file with specifications of containers, ports, volumes, etc.

Deployments

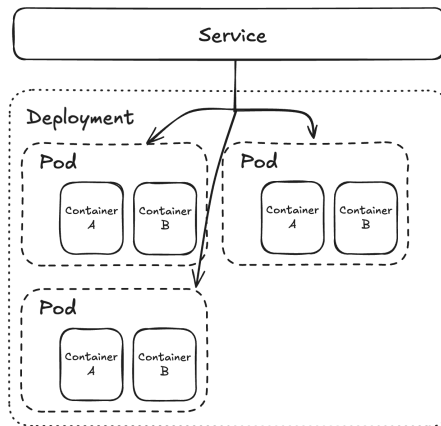
1. Deployments provide a high-level abstraction for managing pods using ReplicaSets



2. ReplicaSets are copies of the pods running, each having the same state as the others
3. It's this resource that provides Autohealing, Autoscaling, & zero downtime updates (updates pods without service interruption or termination)
4. If a pod gets deleted, deployments add another pod automatically, thus countering the ephemeral state of pods

Services

1. Services provide a stable IP address and DNS names to pods



2. The problem with deployments is that if one of the pods gets replaced, its IP address also changes, and thus, services help us to provide a label/selector to connect to the replaced pods without any network errors
3. Services overall provide load-balancing service discovery (label/selectors) and external exposures (access to the pods in different ways)
4. External exposure:
 - a. ClusterIP (default): Used for internal components of the cluster to communicate/access pods
 - b. NodePort (needs to be specified): Used for internal access with NodeIP. Used for testing and development
 - c. LoadBalancer (used by cloud provider): Exposes the pods over the internet using the cloud provider's load balancer (example: Amazon Load Balancer (ALB))

StatefulSet

1. A stateful set is a resource that manages stateful applications like databases.
2. Provides guarantees about orders and the uniqueness of pods

Jobs

1. Jobs are tasks that run one or more pods to completion of a task
2. Example: Running an `init_db.sql` or `migrations.sql` script for installing the prerequisites of a database.

CronJobs

1. This resource runs scheduled jobs at specified times or intervals
2. Example: Running a task for users who are inactive for more than 6 months every month.

ConfigMaps

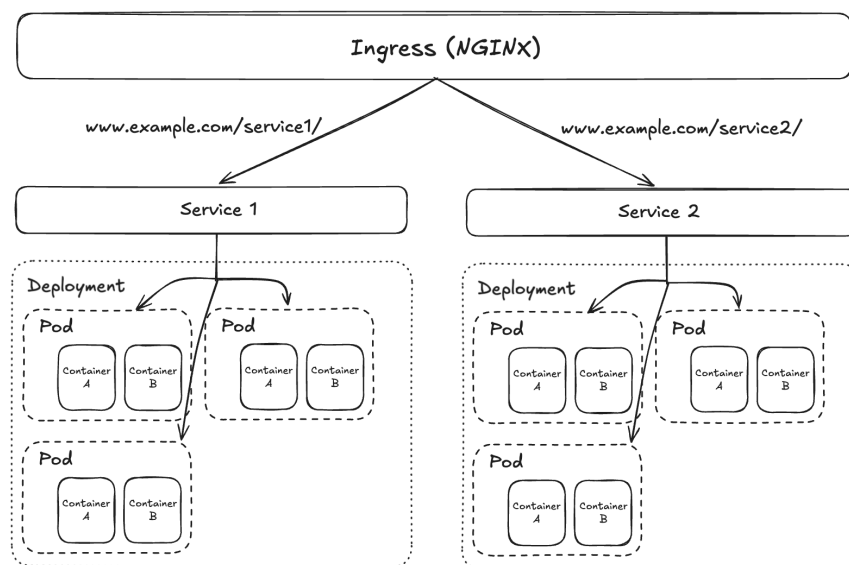
1. Stores configuration data that can be consumed by the pods
2. Example: Defining environment variables for a backend application

Secret

1. Manages sensitive information like passwords, secret keys, and OAuth tokens
2. Example: Stores a private key that can be used to decode a JSON token for Authentication

Ingress

1. Problems with the service resources are that load-balancing isn't flexible for changes.
Example: Controlling the traffic based on URL path, Domain, TLS termination, etc., is not supported
2. To counter these issues, an ingress controller provides
 - a. Path-based and host-based routing
 - b. TLS termination
 - c. Sticky sessions
 - d. IP whitelisting/blacklisting
 - e. Saves cost by only providing a single entry point for multiple services instead of using multiple cloud load balancers
 - f. Advanced traffic control, like rate limiting, load balancing strategies, etc
3. Example: an NGINX ingress controller provides features like path-based routing(separate for front-end & backend), reverse-proxy, rate-limiting, API-gateway, etc



RBAC (Role-based Access Control)

1. RBAC is a resource that secures clusters by defining
 - a. Users and groups (externally using LDAP, IAM, OAuth)
 - b. Service accounts (for pods, cluster controllers, etc)
 - c. Roles and cluster roles (within namespaces/cluster)
 - d. Rolebinding & ClusterRolebindings (roles to users mapping)
2. RBAC ensures least privilege access and protects critical resources like ConfigMaps and Secrets from unauthorized access
3. Admins can define roles for dev, QA, ops, etc